

FRE-7773 Final Project: Deep Momentum Networks for Time-Series Momentum

Shunjie Zhu
sz3571

May 12, 2026

Abstract

This report studies the paper *Enhancing Time Series Momentum Strategies Using Deep Neural Networks* by Lim, Zohren, and Roberts. The paper’s main idea is simple: instead of using machine learning only to predict whether the market goes up or down, train a neural network to directly choose the trading position. The model output is a position between -1 and +1, and the trading return is volatility-scaled so that each asset targets a stable risk level. I implement a simplified Deep Momentum Network (DMN) in Python using a small neural network trained with a Sharpe-ratio objective. I validate the approach on real market data using SPY as a single-asset example and a multi-asset equity panel consisting of SPY, AAPL, ADBE, and AMD. The empirical design uses walk-forward out-of-sample testing, benchmark strategies, risk-adjusted performance metrics, and transaction cost sensitivity analysis. The goal is not to prove that neural networks always win, but to test the paper’s idea in a credible and transparent way.

1 Paper and Motivation

The selected paper is *Enhancing Time Series Momentum Strategies Using Deep Neural Networks* by Bryan Lim, Stefan Zohren, and Stephen Roberts. The paper studies time-series momentum, which is one of the oldest and most intuitive trading ideas:

If an asset has been trending up, go long. If it has been trending down, go short.

Traditional time-series momentum strategies usually require manual design choices. For example, we must decide whether to use a 12-month return signal, a moving-average crossover, or a MACD indicator. We must also decide how large the long or short position should be. The paper’s contribution is to replace these hand-designed rules with a neural network that learns the position directly from historical features.

The paper’s central research question can be summarized as:

Can a neural network that directly outputs trading positions improve time-series momentum performance?

This is different from the usual “predict tomorrow’s return” approach. In finance, prediction accuracy is not the final goal. A model with decent prediction accuracy can still produce a bad trading strategy if it takes too much risk, trades too often, or loses money after transaction costs. The DMN framework therefore trains the model closer to the final trading objective.

2 Main Idea in Simple Terms

The model sees information available at time t , such as recent returns, volatility, MACD trend signals, and interest-rate variables. It then outputs a number

$$X_t \in [-1, 1].$$

This number is the trading position:

- $X_t = 1$: fully long;
- $X_t = -1$: fully short;
- $X_t = 0$: no position;
- values between -1 and 1: partial exposure.

The strategy return is:

$$R_{t+1} = X_t \cdot \frac{\sigma_{\text{target}}}{\sigma_t} \cdot r_{t+1},$$

where r_{t+1} is the next-period asset return, σ_t is the estimated annualized volatility at time t , and σ_{target} is the target annual volatility. In the notebook, I use a 15% annual volatility target.

The most important training objective is the negative Sharpe ratio:

$$\mathcal{L}(\theta) = -\frac{\sqrt{252} \bar{R}}{\sqrt{\frac{1}{T} \sum_{t=1}^T (R_t - \bar{R})^2}}.$$

Minimizing this loss means maximizing risk-adjusted trading performance. This aligns the machine-learning model with the investment objective.

3 Implementation Summary

The full implementation is in:

`Final_Project/notebooks/Deep_Momentum_Network_SPY.ipynb`.

The notebook implements a lightweight version of the paper's method:

- A one-hidden-layer neural network with a tanh output, so positions are bounded in $[-1, 1]$.
- A custom NumPy/Adam optimizer.
- A Sharpe-ratio objective for the main DMN model.
- Additional simplified objectives for comparison, including average return, MSE-style signal fitting, and binary direction.
- Volatility scaling to a 15% target annual volatility.
- Walk-forward validation to avoid look-ahead bias.

3.1 Features

The features follow the paper’s structure. For each asset, I compute:

- normalized 1-day return;
- normalized 1-month, 3-month, 6-month, and 12-month returns;
- MACD-style trend indicators: 8/24, 16/48, and 32/96;
- realized volatility and normalized volatility;
- daily Federal Funds Rate from `DFF.csv`;
- monthly Federal Funds Rate from `FEDFUNDS.csv`.

The feature construction uses only past information. The target return is the next-day return, so the model is always trained to choose a position at time t before observing r_{t+1} .

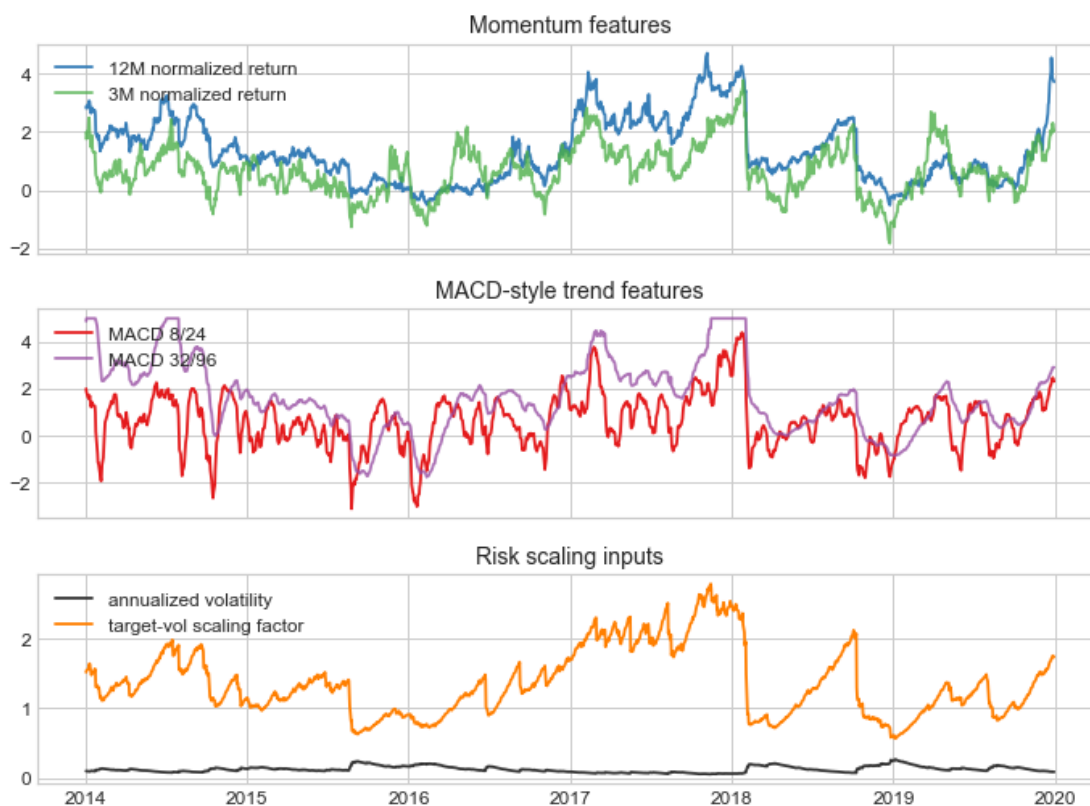


Figure 1: Examples of momentum, MACD, and volatility features used by the model.

4 Validation Design

The final-project instruction asks us to validate the efficacy of the approach on real data. For this paper, the validation question is:

Does a neural-network position model improve time-series momentum performance compared with traditional rules?

I use two empirical tests.

4.1 Single-Asset SPY Test

The first experiment uses SPY as a clean single-asset example. This is useful because SPY is liquid, easy to understand, and familiar to most classmates. The SPY test compares:

- Long Only;
- Sign(Return) Momentum;
- MACD Momentum;
- Linear Sharpe model;
- MLP models trained with Sharpe, average return, MSE, and binary objectives.

4.2 Multi-Asset / Equity-Panel Real-Data Test

The paper evaluates a diversified futures universe. That exact futures database is not freely available in this project. To build a more paper-aligned validation, I add a multi-asset real-data panel using:

Asset	Description
SPY	Broad U.S. equity market ETF
AAPL	Apple Inc.
ADBE	Adobe Inc.
AMD	Advanced Micro Devices, Inc.

This is not as diversified as the paper’s 88 futures contracts, but it is a stronger validation than a single asset because the model must learn across several return series. The raw data window is 2012–2019. Because 12-month momentum and rolling volatility features require lookback windows, the usable feature panel begins in early 2013.

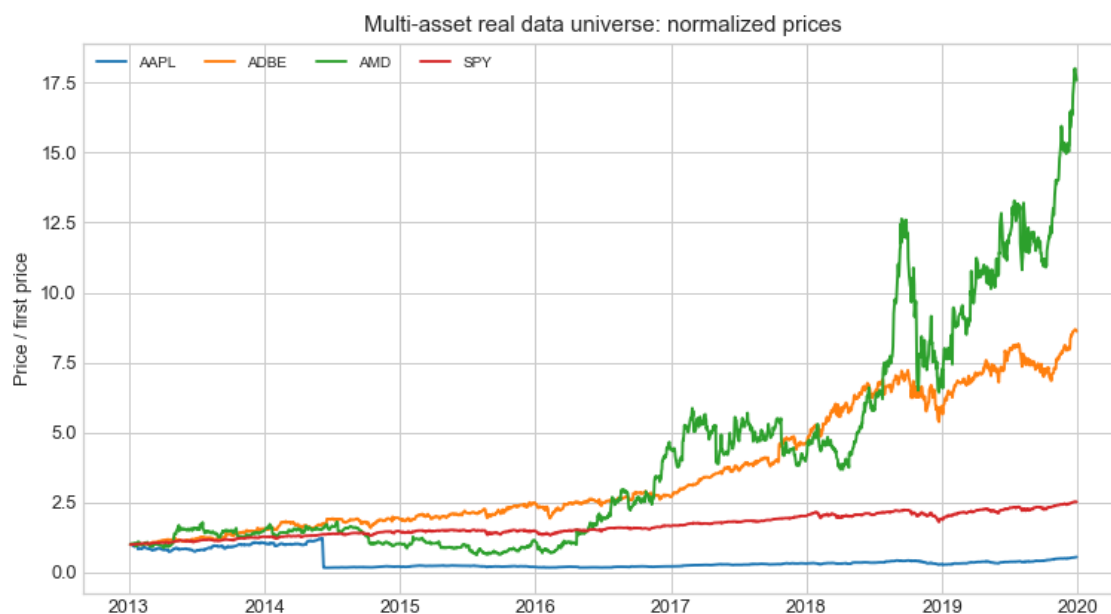


Figure 2: Normalized prices for the multi-asset validation universe.

4.3 Out-of-Sample Walk-Forward Procedure

The multi-asset test uses an expanding walk-forward design:

Fold	Training Period	Validation Period	Test Period
1	2012–2014	2015	2016
2	2012–2015	2016	2017
3	2012–2016	2017	2018
4	2012–2017	2018	2019

This design is important because financial data should not be randomly shuffled. The model is trained only on past data, tuned on a later validation year, and evaluated on a future unseen test year. This avoids look-ahead bias.

5 Benchmarks and Metrics

The benchmarks are chosen to match the spirit of the paper:

- **Long Only:** always hold the asset with volatility scaling.

- **Sign Momentum:** go long if the 12-month return is positive and short if it is negative.
- **MACD Momentum:** use a traditional moving-average trend signal.
- **DMN-Sharpe:** neural network trained to maximize Sharpe ratio.
- **DMN-Average Return:** neural network trained to maximize average strategy return.

The performance metrics are:

- annualized return;
- annualized volatility;
- Sharpe ratio;
- Sortino ratio;
- maximum drawdown;
- Calmar ratio;
- fraction of positive return days;
- average profit divided by average loss;
- turnover;
- average absolute position.

Transaction costs are also tested at 0, 1, 2, 3, and 5 basis points per unit of turnover. This is important because neural-network strategies can trade more frequently than simple benchmark rules.

6 Single-Asset SPY Results

The SPY-only experiment is useful as a controlled demonstration. However, SPY from 2015 to 2019 is mostly a bull-market period, so long-only SPY is a very strong benchmark.

Strategy	Ann. Ret.	Ann. Vol.	Sharpe	MDD	Calmar	Turnover
Long Only	14.33%	15.54%	0.92	-22.09%	0.65	0.00%
Sign(Return)	6.19%	15.57%	0.40	-36.23%	0.17	5.41%
MACD	0.90%	8.90%	0.10	-17.39%	0.05	4.27%
MLP-Sharpe	0.37%	2.94%	0.12	-5.25%	0.07	5.76%
MLP-AvgRet	4.75%	10.25%	0.46	-21.71%	0.22	1.24%
MLP-Binary	1.35%	2.37%	0.57	-3.78%	0.36	3.03%

Table 1: SPY out-of-sample results after 1 bp turnover cost.

Paper-style Exhibit: Cumulative Returns Rescaled to 15% Target Volatility

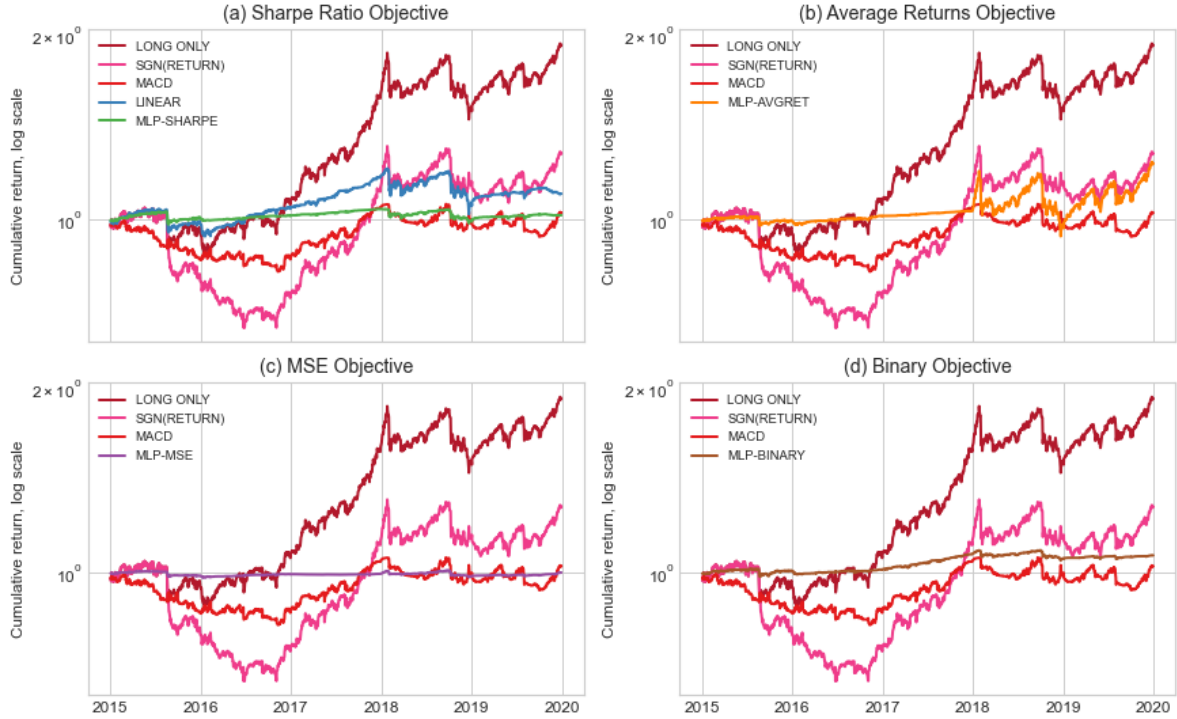


Figure 3: Paper-style cumulative return comparison for the SPY experiment.

The SPY result shows an important limitation: on a single equity ETF during a strong bull market, the best strategy can simply be long-only exposure. This does not invalidate the paper's idea. It shows why the original paper emphasizes diversified multi-asset testing.

7 Multi-Asset / Equity-Panel Real-Data Results

The multi-asset experiment is the more important validation block because it is closer to the paper’s empirical setup. Portfolio returns are equal-weighted across the four assets.

Strategy	Ann. Ret.	Ann. Vol.	Sharpe	Sortino	MDD	Calmar	Turnover	Avg. Pos.
Long Only	21.47%	11.98%	1.79	2.57	-15.46%	1.39	0.00%	100.00%
Sign Momentum	15.54%	10.39%	1.50	2.12	-12.82%	1.21	3.93%	77.24%
MACD Momentum	6.10%	6.23%	0.98	1.35	-6.52%	0.94	2.87%	47.60%
DMN-Sharpe	8.28%	5.12%	1.62	2.35	-4.64%	1.78	4.36%	36.96%
DMN-AvgRet	11.08%	7.97%	1.39	2.01	-10.41%	1.06	4.32%	51.64%

Table 2: Multi-asset out-of-sample results, 2016–2019 test years, after 1 bp turnover cost.

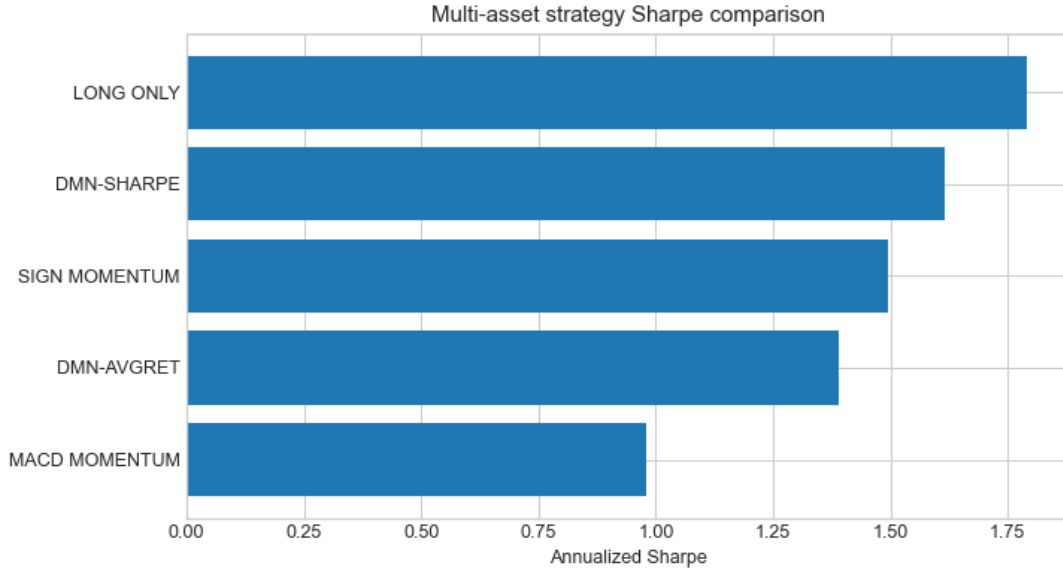


Figure 4: Sharpe ratio comparison for the multi-asset validation.

The multi-asset results are more encouraging for the DMN idea. Long-only remains strong because the test universe is equity-heavy and the period includes a strong equity market. However, DMN-Sharpe improves over MACD Momentum and produces a higher Sharpe ratio than Sign Momentum after controlling for volatility and drawdown. DMN-Sharpe also has the smallest maximum drawdown in the table.

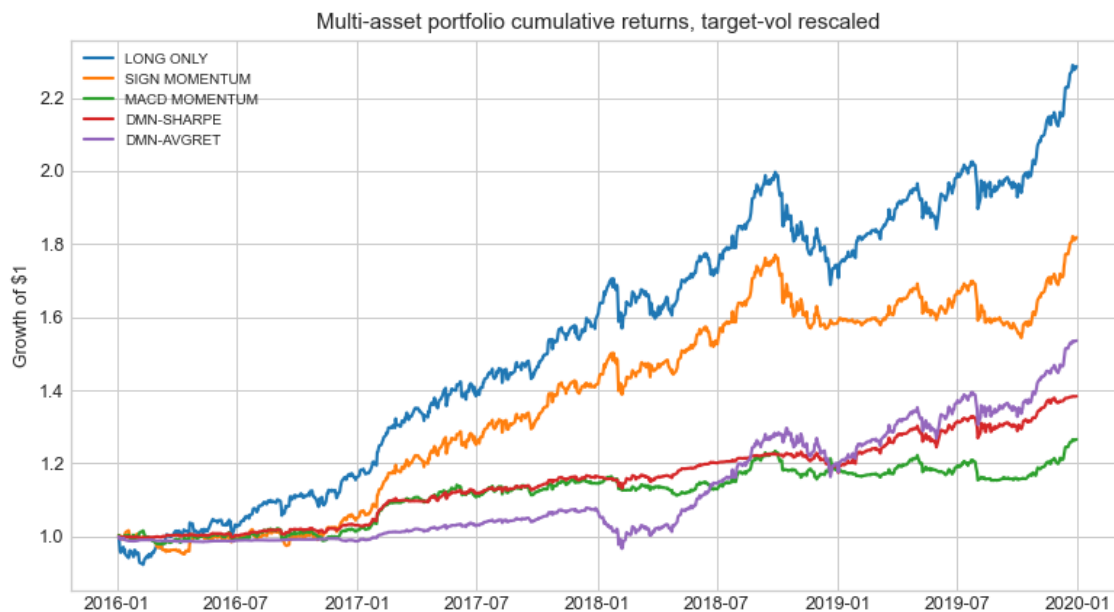


Figure 5: Cumulative returns for the multi-asset validation.

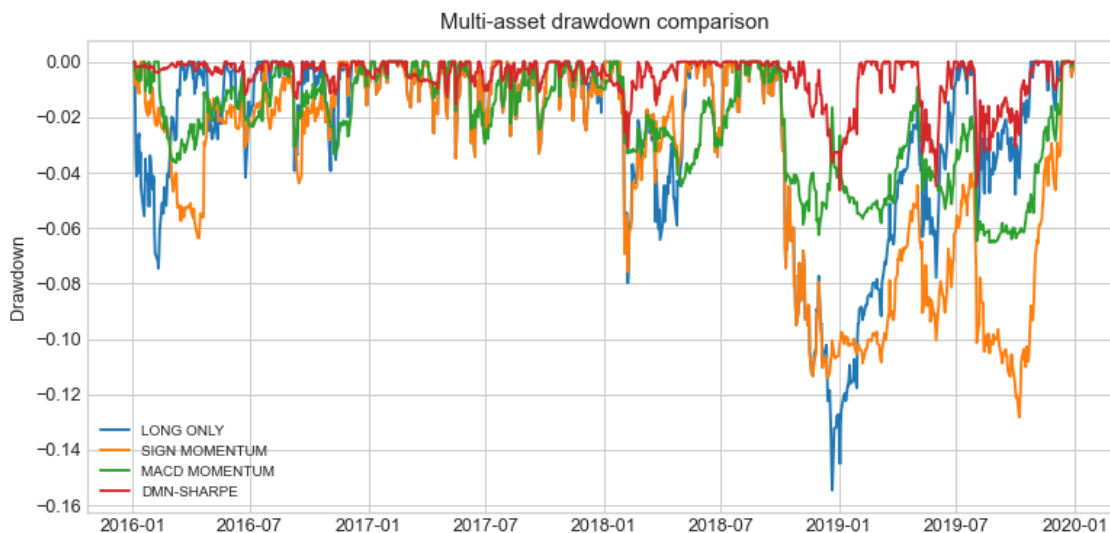


Figure 6: Drawdown comparison for the multi-asset validation.

8 Transaction Cost Sensitivity

Transaction costs matter because machine-learning models may change positions more frequently than simple benchmark strategies. Net returns are computed as:

$$\tilde{R}_{t+1} = R_{t+1} - c \cdot |w_t - w_{t-1}|,$$

where c is the transaction cost in decimal form and $|w_t - w_{t-1}|$ is turnover.

Strategy	0 bps	1 bps	2 bps	3 bps	5 bps
Long Only	1.79	1.79	1.79	1.79	1.79
Sign Momentum	1.51	1.50	1.49	1.48	1.46
MACD Momentum	1.00	0.98	0.96	0.95	0.91
DMN-Sharpe	1.64	1.62	1.59	1.57	1.52
DMN-AvgRet	1.41	1.39	1.38	1.36	1.33

Table 3: Sharpe ratio under different transaction cost assumptions.

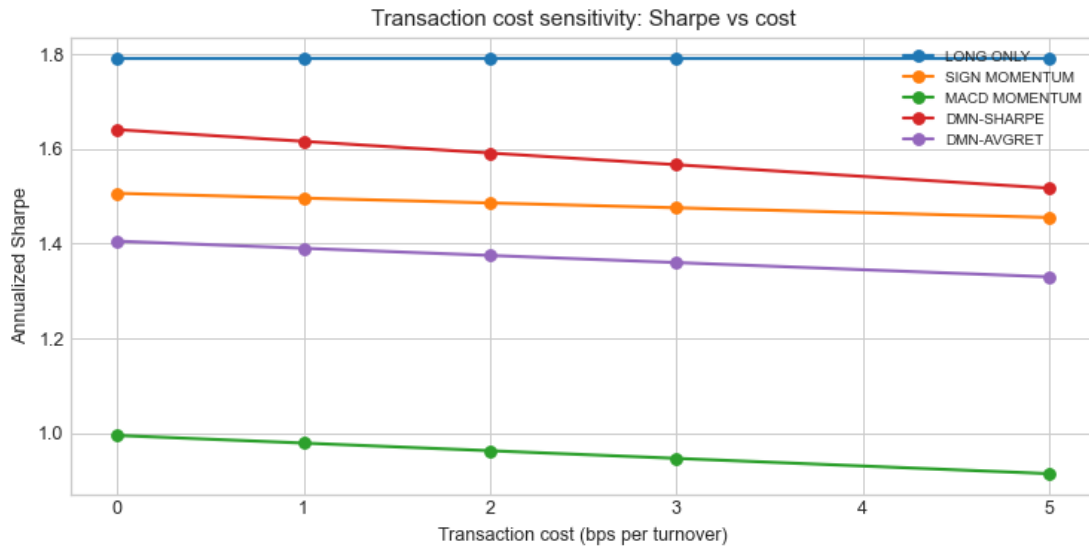


Figure 7: Transaction cost sensitivity: Sharpe ratio versus transaction cost.

DMN-Sharpe remains reasonably robust from 0 to 5 bps. Its Sharpe declines from 1.64 to 1.52, which shows that turnover matters but does not fully remove the strategy’s risk-adjusted performance. This is aligned with the paper’s emphasis that transaction costs are a key practical issue for DMN strategies.

9 Position Behavior and Cross-Asset Distribution

One useful feature of the DMN approach is that the model does not only say “buy” or “sell.” It can choose a partial position. This makes the output easier to interpret as position sizing.

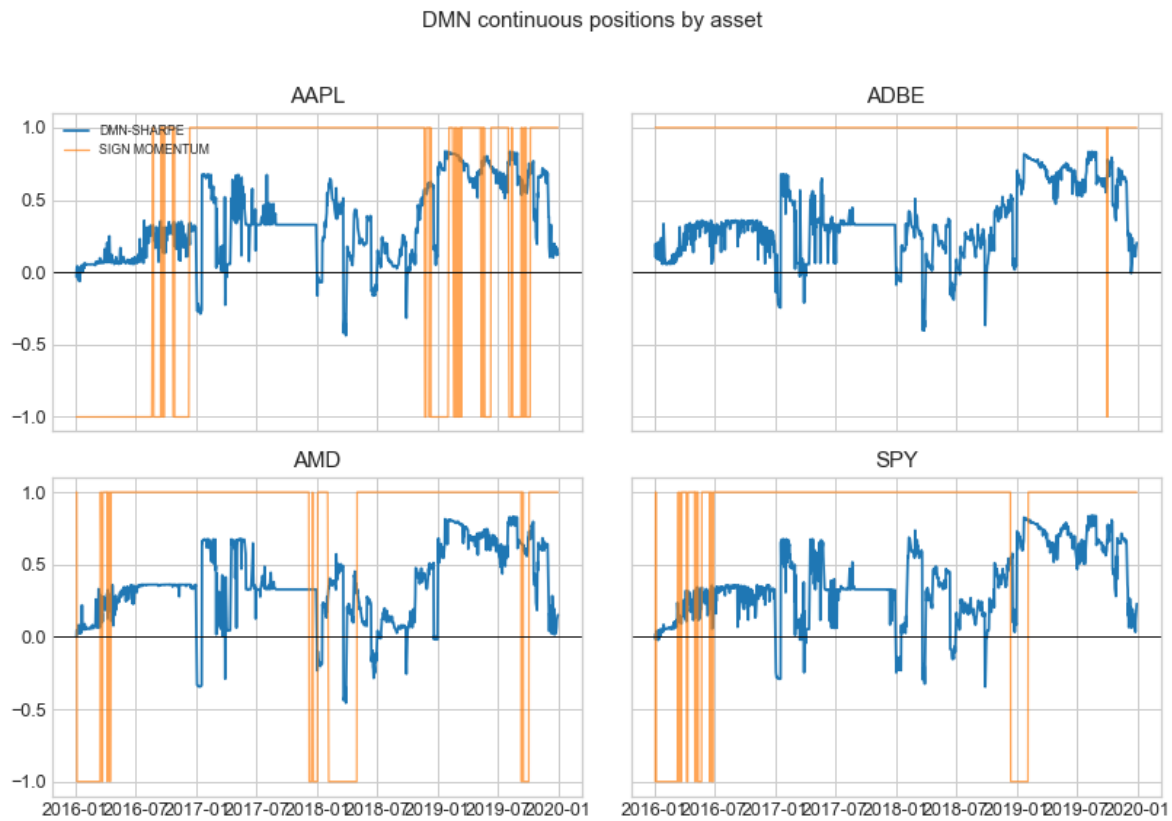


Figure 8: DMN-Sharpe positions compared with the hard sign-momentum rule.

The paper also studies whether performance is consistent across assets. The boxplot below shows the distribution of per-asset/year Sharpe ratios.

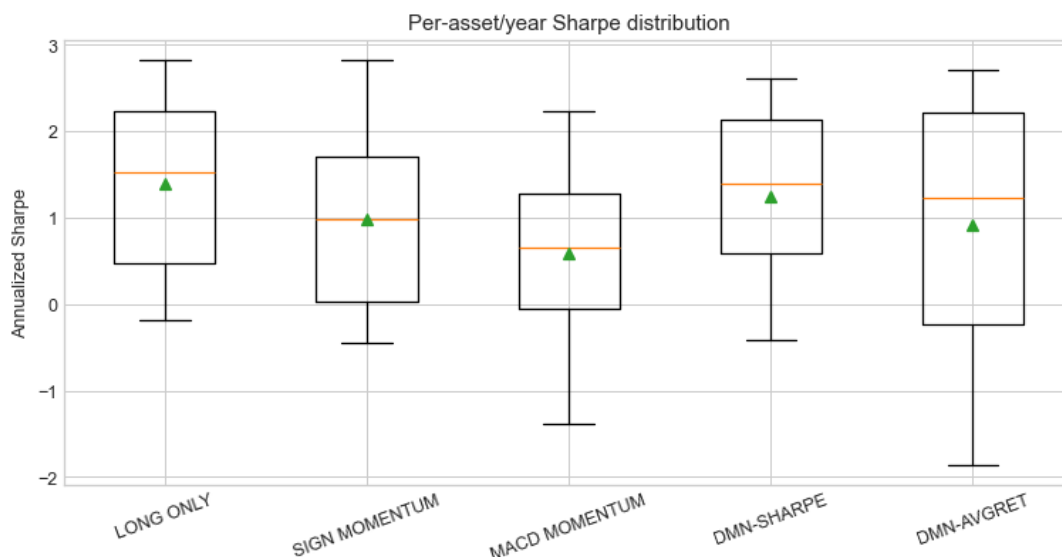


Figure 9: Per-asset/year Sharpe distribution across the multi-asset panel.

Figure 9 shows that DMN-Sharpe has a relatively high median Sharpe and a strong upper-quartile performance, but its distribution is still wide. This means the model is not uniformly better on every asset-year. Its value comes from risk-controlled position sizing rather than guaranteed outperformance on each individual asset.

10 Interpretation

The results should be interpreted carefully. The notebook is not a full replication of the paper because:

- the paper uses 88 continuous futures contracts, while this project uses SPY and three individual stocks;
- the paper tests deeper architectures such as LSTM and WaveNet, while this project uses a compact MLP;
- the available multi-asset universe is equity-heavy, so long-only exposure is a strong benchmark during the test period.

Even with these limitations, the validation is useful. It demonstrates that:

- the DMN position-output framework can be implemented in code;

- the model can be trained directly on a trading objective;
- the strategy can be tested out of sample without look-ahead bias;
- performance can be compared against long-only, sign-momentum, and MACD benchmarks;
- risk-adjusted metrics and transaction costs materially affect the conclusion.

The main empirical takeaway is:

DMN-Sharpe does not dominate every benchmark, but it improves over traditional MACD momentum and provides a smoother, risk-controlled position-sizing rule. This supports the paper’s central idea that learning positions directly is a meaningful alternative to manually designed trend signals.

11 Conclusion

This project implements and validates the core idea of Deep Momentum Networks. The paper argues that neural networks can enhance time-series momentum by learning both trend estimation and position sizing. My notebook implements this idea using real market data and tests it with an out-of-sample walk-forward framework.

The strongest evidence from my validation is not that DMNs always beat long-only exposure. Instead, the strongest evidence is that the DMN framework creates an interpretable, risk-scaled position model that can outperform traditional momentum rules such as MACD while keeping drawdowns controlled. This is a credible small-scale test of the paper’s approach and a useful foundation for explaining the method to classmates.

References

- Lim, B., Zohren, S., and Roberts, S. (2019). *Enhancing Time Series Momentum Strategies Using Deep Neural Networks*. The Journal of Financial Data Science.
- Moskowitz, T., Ooi, Y. H., and Pedersen, L. H. (2012). *Time Series Momentum*. Journal of Financial Economics.
- Yahoo Finance or data source note for SPY/AAPL/ADBE/AMD.
- Federal Reserve Economic Data (FRED) for DFF and FEDFUNDS.